ChillFlix — Découpage en tâches et organisation du travail

Nov 12, 2026 · @Ruben

Sprint 1 en cours dans iceScrum (10.11 → 30.11.2026) : périmètre réduit aux films, 31 tâches détaillées, 29 tests d'acceptation. Plan validé par l'équipe le 12.11.2026. Basé sur le cadrage initial (CPNV, 03.11.2026), la maquette, les wireframes, le daily du 10.11 et les user stories d'iceScrum.

1. Réponse courte : peut-on travailler à 3 sur la même page ?

Oui, et c'est la bonne façon de faire en Vue. Une page (une *vue*) n'est qu'un assemblage de composants. Exemple pour la page de résultats :

```
<!-- src/views/SearchView.vue : fichier court, qui ne fait qu'assembler -->
<template>
  <ResultsHeader />
  <SearchFilters />
  <ActiveFilters />
  <ResultsGrid />
  <BasePagination />
</template>
```

Chaque composant est un fichier `.vue` séparé, avec un seul propriétaire et une seule branche Git, donc il n'y a pas de conflit. Trois règles pour que ça marche :

1. **La vue est créée en premier, avec des composants vides** (un simple `<div>ResultsGrid</div>`). Ensuite chacun remplit le sien sans toucher à la vue.
2. **Les données partagées passent par un store Pinia**, pas par des props entre composants frères. `SearchFilters` écrit `store.genre`, `ResultsGrid` lit `store.results`. Les deux composants n'ont pas besoin de se connaître : il suffit de se mettre d'accord sur la forme du store (section 3).
3. **Fausses données pendant le développement** : un fichier `src/mocks/` avec une vraie réponse TMDB copiée en JSON. Celui qui fait l'UI n'attend pas que l'appel API soit prêt.

2. Architecture des fichiers (à créer dans le sprint en cours)

```
src/
  api/
    tmdb.js            # instance axios (baseURL, token, language=fr-FR)
    search.js          # searchMulti, discover
    movies.js          # getMovie(id)
    tv.js              # getTv(id), getSeason(id, n)
    people.js          # getPerson(id)
    genres.js          # getMovieGenres, getTvGenres
  stores/
    search.js          # requête, type, filtres, tri, page, résultats,
loading, error
    genres.js          # listes de genres chargées une fois
  router/index.js      # toutes les routes, créées en une fois
  views/               # pages = assemblage, presque pas de logique
    HomeView.vue  SearchView.vue  MovieView.vue  TvView.vue  PersonView.vue
NotFoundView.vue
  components/
    layout/   AppHeader.vue  AppFooter.vue  SearchBar.vue
    ui/       BaseButton.vue  BaseSelect.vue  FilterChip.vue  MediaCard.vue
CardSkeleton.vue
              BasePagination.vue  BaseTabs.vue  BaseCarousel.vue
EmptyState.vue  ErrorState.vue
    search/   ResultsHeader.vue  SearchFilters.vue  ActiveFilters.vue
ResultsGrid.vue  PersonCard.vue
    detail/   DetailHero.vue  DetailActions.vue  Breadcrumb.vue
OverviewSection.vue
              CastCarousel.vue  RecommendationsCarousel.vue
TrailerPlayer.vue
    tv/       SeasonsList.vue  EpisodesList.vue
    person/   PersonHeader.vue  Filmography.vue
  assets/tokens.css    # variables --cf-* reprises de la maquette
  mocks/               # réponses TMDB en JSON pour développer sans API
```

Routes (série et personne en bonus) : `/`, `/recherche?q=&type=&genre=&annee=&tri=&page=`, `/film/:id`, `/serie/:id`, `/personne/:id` et une page 404. Les filtres et le tri sont dans l'URL (demandé par la maquette), ce qui règle aussi « Retour aux résultats avec les filtres conservés ».

3. Contrats à fixer avant de coder (30 min ensemble)

C'est ce qui permet de travailler en parallèle. À écrire dans le README ou dans `design.md` :

- **Forme d'un résultat normalisé**, utilisée par toutes les cartes : `{ id, type: 'movie' | 'tv' | 'person', title, year, posterPath, rating, popularity }`. Le service API convertit la réponse TMDB dans ce format : les composants ne voient jamais le format brut.
- **Store `search`** : `query`, `type`, `genre`, `year`, `sort`, `page`, `results`, `totalResults`, `loading`, `error`, et les actions `fetchResults()`, `resetFilters()`.
- **Props de `MediaCard`** : `item` (le résultat normalisé). La carte émet un lien vers `/film/:id`, `/serie/:id` ou `/personne/:id` selon `type`.
- **Props de `BaseCarousel`** : `items` plus un slot pour la carte. Il est réutilisé par la distribution et par les recommandations.

4. Workflow Git

- Une tâche iceScrum = une branche `feature/<us>-<sujet>` créée depuis `develop` (ex. `feature/fiche-film-hero`).
- Petites pull requests vers `develop`, relues par un autre membre avant le merge.
- `git pull origin develop` chaque matin, puis rebase ou merge de `develop` dans votre branche.
- Les fichiers « carrefour » (`router/index.js`, `App.vue`, `main.js`, `tokens.css`, les stores) sont créés une fois pour toutes dans les Fondations. Ensuite, toute modification de ces fichiers est annoncée au daily.
- Clé TMDB dans un `.env.local` (non commité) ; un `.env.example` est commité avec `VITE_TMDB_TOKEN=`.

5. Les tâches dans iceScrum

31 tâches sont dans iceScrum depuis le 12.11.2026, rattachées aux user stories. La liste complète par sprint est dans les tableaux de la section 6.

- **Titre** : code, action et personne, par exemple « R6 · Carte de film et carte de chargement — Sellé ».
- **Couleur** : une par personne. Ruben en bleu, Arthur en vert, Sellé en violet.
- **Description** en 6 rubriques : Pourquoi, Fichiers, Étapes numérotées, Aide (lien vers la doc), Dépend de, Fini quand. « Fini quand » cite les tests d'acceptation que la tâche doit faire passer.
- **Estimation** : en heures.
- **Rattachement** : Rechercher porte les fondations (F1–F6), la recherche (R2–R9) et la finalisation (Z1–Z2) ; Fiche film porte M1–M8 ; Filtrer porte FI1–FI5 ; Trier porte T1–T2. La tâche « Documentation API TMDB » d'Arthur reste telle quelle.

- **Tableau du sprint 1** : il affiche aussi M3–M8, Z1 et Z2, car iceScrum met une user story entière dans un sprint. Ces tâches sont marquées « Sprint 2 » : les laisser dans « À faire ».

Comment utiliser une tâche :

1. Te mettre responsable de la tâche et la passer en « En cours ».
2. Créer la branche `feature/<code>-<sujet>` depuis `develop` (ex. `feature/r6-media-card`).
3. Coder jusqu'au « Fini quand », puis ouvrir une pull request vers `develop`.
4. Après relecture et merge, passer la tâche en « Terminé ».

Par où commencer : F1 à F6 d'abord, car elles débloquent le reste. Pendant ce temps, Arthur fait R2 et M1, qui ne dépendent de rien.

Bonus, sans tâche pour l'instant : fiche série, fiche personne, panneau de filtres mobile, « Charger plus » sur mobile. À créer seulement si le sprint 2 est fini en avance.

6. Planning réel : 2 sprints de 3 semaines

Il reste environ 22 h de développement par sprint pour l'équipe, d'où le périmètre réduit ci-dessous. Calcul : 4 périodes × 6 semaines = 24 périodes, soit 18 h par personne (périodes de 45 min) et 54 h pour l'équipe. Sans le planning, la review et la rétro (environ 2 périodes par sprint), il reste environ 15 h par personne, soit 45 h pour l'équipe.

Ce qui sort du périmètre (validé par l'équipe le 12.11, à confirmer avec le PO ; le cadrage ne demande que recherche, vue détaillée, filtres et tri) :

- US Fiche série et US Fiche personne (−12 h), gardées en bonus si vous avez de l'avance. Conséquence : la recherche se limite aux films (`/search/movie` au lieu de `/search/multi`), le filtre Type disparaît et les acteurs de la distribution ne sont pas cliquables. Les tests des US Rechercher, Filtrer et Trier qui parlent de séries ou de personnes sont à retirer.
- Panneau de filtres mobile et « Charger plus » mobile : sur mobile, les filtres s'empilent et la pagination reste (−2,5 h).
- Onglets synchronisés avec le défilement : de simples ancres suffisent (−1 h).
- Estimations resserrées sur le reste.

Sprint 1 (semaines 1 à 3) : chercher un film et ouvrir sa fiche (≈ 22 h)

Tâche (code iceScrum)	Est.	Qui
F1 Couleurs et polices du site (`tokens.css`)	1 h	Sellé

Tâche (code iceScrum)	Est.	Qui
F2 Connexion à l'API TMDB (axios, `.env.example`)	1 h	Ruben
F3 Toutes les pages vides et leurs adresses	1,5 h	Ruben
F4 Header et footer	2,5 h	Sellé
F5 Store de la recherche + fausses données	1 h	Ruben
F6 `design.md` + règles de l'équipe	1 h	Sellé
R2 Requête de recherche + format commun	1,5 h	Arthur
R3 Lancer la recherche, tout garder dans l'URL	2,5 h	Ruben
R4 Barre de recherche	1,5 h	Ruben
R5 Titre de la page de résultats	0,5 h	Arthur
R6 Carte de film + carte de chargement	2 h	Sellé
R7 Grille de résultats	1 h	Sellé
R8 Pagination	1 h	Arthur
R9 Messages « Aucun résultat » et « Erreur »	1 h	Arthur
M1 Récupérer les infos d'un film	1,5 h	Arthur
M2 Page fiche film : chargement, erreur, introuvable	1,5 h	Arthur

Charge : Ruben 7,5 h, Sellé 7,5 h, Arthur 7 h.

Sprint 2 (semaines 4 à 6) : fiche complète, filtres, tri, rendu (≈ 22 h)

Tâche (code iceScrum)	Est.	Qui
M3 Haut de la fiche film	3 h	Arthur
M4 Bouton Retour et fil d'Ariane	1 h	Arthur
M5 Onglets de la fiche	1 h	Sellé
M6 Section Aperçu et bande-annonce	1,5 h	Arthur
M7 Carrousel réutilisable	1,5 h	Sellé
M8 Carrousels Distribution et Recommandations	1,5 h	Sellé
FI1 Charger la liste des genres	0,5 h	Ruben

Tâche (code iceScrum)	Est.	Qui
FI2 Liste déroulante réutilisable	1 h	Sellé
FI3 Zone des filtres	2 h	Ruben
FI4 Faire marcher les filtres avec l'API	2 h	Ruben
FI5 Filtres actifs (pastilles)	1,5 h	Sellé
T1 Choix du tri et tri sans texte	1,5 h	Ruben
T2 Tri avec texte, tri gardé entre les pages	1,5 h	Ruben
Z1 Vérifier l'affichage ordi / tablette / mobile	1 h	Arthur
Z2 README final et quelques tests	1,5 h	Arthur

Charge : Ruben 7,5 h, Arthur 8 h, Sellé 6,5 h. La démo se prépare ensemble sur le temps de la review finale.

Règle de sécurité : fin de la semaine 5, gel des fonctionnalités. Ensuite, uniquement corrections, README et démo. Si une tâche du sprint 1 déborde, ce sont les tâches du sprint 2 les moins prioritaires (carrousels, onglets) qui sautent, pas les filtres ni le tri, qui sont exigés par le cadrage.

7. Pourquoi cette répartition

Chacun garde sa zone de fichiers, ce qui limite les conflits Git : Ruben la logique (API de recherche, stores, URL), Arthur les fiches (il connaît déjà TMDB), Sellé les composants visuels réutilisables (cohérence avec sa maquette). Répartition validée par l'équipe le 12.11.2026.

8. Suivi et points ouverts

Fait le 12.11.2026 :

- Périmètre réduit aux films et répartition validés par l'équipe.
- iceScrum : user stories Rechercher (5 pts), Fiche film (8), Filtrer (5) et Trier (3) acceptées et estimées ; release « ChillFlix v1 » (10.11 → 21.12) avec sprint 1 (10.11 → 30.11, en cours, contient Rechercher et Fiche film) et sprint 2 (1.12 → 21.12).
- 31 tâches détaillées et colorées par personne.
- Tests d'acceptation relus : 16 corrigés pour ne parler que des films, 10 hors périmètre supprimés (séries, personnes, filtre Type, doublon « Retour à la liste »). Il en reste 29 : Rechercher 7, Fiche film 10, Filtrer 5, Trier 7. Descriptions de Rechercher et Filtrer

limitées aux films.

- README sans « collection personnelle », nom de Sellé corrigé dans le daily du 10.11.

À faire :

- [ ] Chacun se met responsable de ses tâches dans iceScrum en les commençant.
- [ ] Début du sprint 2 : y planifier Filtrer et Trier.
- [ ] Commiter les changements du README et du daily, puis réexporter le PDF du daily du 10.11 (il contient encore « Scellé »).

À régler avec le Product Owner :

- [ ] Confirmer le périmètre films uniquement (séries et personnes en bonus).
- [ ] Aligner la maquette et les US : filtre Note minimale, nom de l'onglet (Recommandations ou Similaires), onglet Avis.
- [ ] Header : retirer « Ma collection » de la maquette, et Films, Découvrir, Nouveautés sauf si une US les couvre.
- [ ] Mettre le responsive et le README dans la Definition of Done.